HW 8 coming soon.

Quiz 11 due next Monday.
Midterm 3: next Tue, Apr 14, 9:30am.

https://en.wikipedia.org/wiki/Logarithmic_spiral#In_nature

Class 23:
Class EXP, Reductions

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

Plan

Complexity

Problems not in **P**

Class **EXP**

Polynomial-time Reductions

Textbook [TCS] Section 13

https://introtcs.org/public/lec_11_running_time.html

Modeling running time

Midterm Review: Computability

RAM is equivalent to One-Tape TM (Class 17)

RAM ≡ Two-Tape-TM ≡ One-Tape-TM

≡ (Ideal) Python ≡ { other program langs}

≡ Conway's Game of Life, Minecraft, (infinite) Minesweeper, (infinite) Magic: The Gathering

All are Turing Complete computation models!

Recap: Computability

Definition (Definition 7.2):

For any Boolean function $F: \{0,1\}^* \to \{0,1\}^*$ and any Turing Machine M, we say M ***computes*** F iff for all $x \in \{0,1\}^*$, $M(x) = F(x)$.

We say F is ***computable*** if and only if there exists a Turing machine M such that computes F.

Rice's Theorem (9.15)

Definition 9.14:

Two machines M_1, M_2 are **functionally equivalent** (denoted $M_1 \equiv M_2$**)**
if $\forall x \in \{0,1\}^*$, $M_1(x) = M_2(x)$

A function $F: \{0,1\}^* \to \{0,1\}$, defined on Turin machines, is **semantic** if for every pair of functionally equivalent TMs (M_1, M_2), $F(M_1) = F(M_2)$.

If F is a sematic property, then either F is **uncomputable**, or F is trivial.

By definition, F is trivial iff for all Turing machine M,

$$F(M) = 0$$
$$F(M) = 1$$

Proof by Reduction

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Proof: If HALT is computable, then NSA is also computable…

1. **<u>Assume</u>** M_{HALT} computes HALT
2. **<u>Construct</u>** $M_{NSA}(x)$
3. **<u>Prove</u>** that M_{NSA} computes NSA (assuming M_{HALT} computes HALT)

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Proving *HALT* is Uncomputable

Assume $HALT$ is computable.

By the assumption (and definition of computable), there exists some TM M_{HALT} that computes $HALT$.

We can use M_{HALT} to build a machine that decides NSA:

$M_{NSA}(x)$: 1. $h = M_{HALT}(x, x)$
2. if $h = 1$: return NOT($\mathcal{U}(x, x) = 1$)
else: 1

Thus, since we know M_{NSA} does not exist, but if we had M_{HALT} we could build it, we have a contradiction! This proves that M_{HALT} must not exist which means $HALT$ is not computable.

Exercise 9.5

Prove that the following function $FINITE : \{0,1\}^* \to \{0,1\}$ is uncomputable. On input $P \in \{0,1\}^*$, we define $FINITE(P) = 1$ if and only if P is a string that represents a ~~NAND++~~ TM program such that there only a finite number of inputs $x \in \{0,1\}^*$ s.t. $P(x) = 1$.

$$\text{Finite}(P) = \begin{cases} 1 & \text{if finite } x \text{ s.t. } P(x) = 1 \\ 0 & \text{o.w.} \end{cases}$$

TM

① Rice's Thm : Finite is semantic nontrivial $\left(\forall M_1 \equiv M_2,\ \text{Finite}(M_1) = \text{Finite}(M_2) \right)$

$\left(\exists M_0\ \text{Finite}(M_0) = 0 \quad \exists M_1\ \text{Finite}(M_1) = 1 \right)$

Recap

Recap: Complexity Class: P

A class for "Polynomial Time"

Definition (13.1):
$TIME_{\text{TM}}(T(n))$ is the set of functions for which a Turing Machine M exists such that M halts after at most $T(n)$ steps for all n-bit input and M computes the function.

$\cancel{O(n^c)}$

$$\text{Class } \mathbf{P} = \bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$$

num bits in inputs

Note: TCS book definition is $\mathbf{P} = \bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(n^c)$

Recap: P is Robust to Machine Definitions

$\mathbf{P}_{TM} = \bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$ One-way infinite tape TM

$\mathbf{P}_{TM2w} = \bigcup_{c \in \mathbb{N}} TIME_{TM2w}(n^c)$ Two-way infinite tape TM

$\mathbf{P}_{\mathrm{RAM}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{RAM}}(n^c)$ **RAM** machine

$\mathbf{P}_{\mathrm{Python}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{Python}}(n^c)$ Idealized **Python** interpreter

$$\mathbf{P}_{TM} = \mathbf{P}_{TM2w} = \mathbf{P}_{\mathrm{RAM}} = \mathbf{P}_{\mathrm{Python}}$$

Functions in P

Functions in P?

$$\mathbf{P}_{TM} = \bigcup_{c\in\mathbb{N}} TIME_{TM}(n^c)$$
$$= \mathbf{P}_{\text{Python}} = \bigcup_{c\in\mathbb{N}} TIME_{\text{Python}}(\boxed{n}^c)$$

bits

$n = m(t+1)$

SORTING

m t-bit

Input: A list of $\boxed{n}$ natural numbers, $x_1, x_2, \boxed{x_3}, \ldots, x_n$.

Output: An ordering of the input list, $x_{i_1}, x_{i_2}, \ldots, x_{i_n}$ where $\{i_1\} \cup \{i_2\} \cup \ldots \{\boxed{i_n}\} = \{1, 2, \ldots, \boxed{n}\}$ and for all $k \in \{1, 2, \ldots, \boxed{n-1}\}$, $x_{i_k} \leq x_{i_{k+1}}$.

What's the running time?

$\Theta(m^2 \cdot (t+1)) \leq n^2$

Yes

Functions in P?

$$\mathbf{P}_{TM} = \bigcup_{c\in\mathbb{N}} TIME_{TM}(n^c)$$
$$= \mathbf{P}_{\text{Python}} = \bigcup_{c\in\mathbb{N}} TIME_{\text{Python}}(n^c)$$

$$HALT(w,x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

What's the running time?

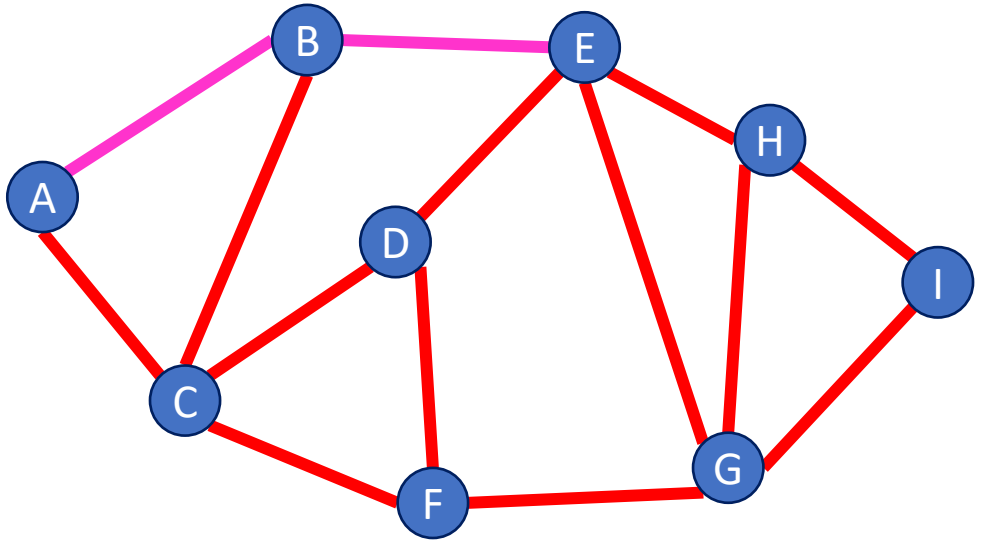
X no time (don't halt)

HALT $\notin$ P

Definition: a ***path*** from v_1 to v_k in a graph $G = (V, E)$ is a sequence of nodes $(v_1, v_2, \ldots, v_k)$ such that $\forall\ 1 \leq i \leq k-1$, $(v_i, v_{i+1}) \in E$. A ***simple path*** is a path with no repeated nodes.

Shortest Path

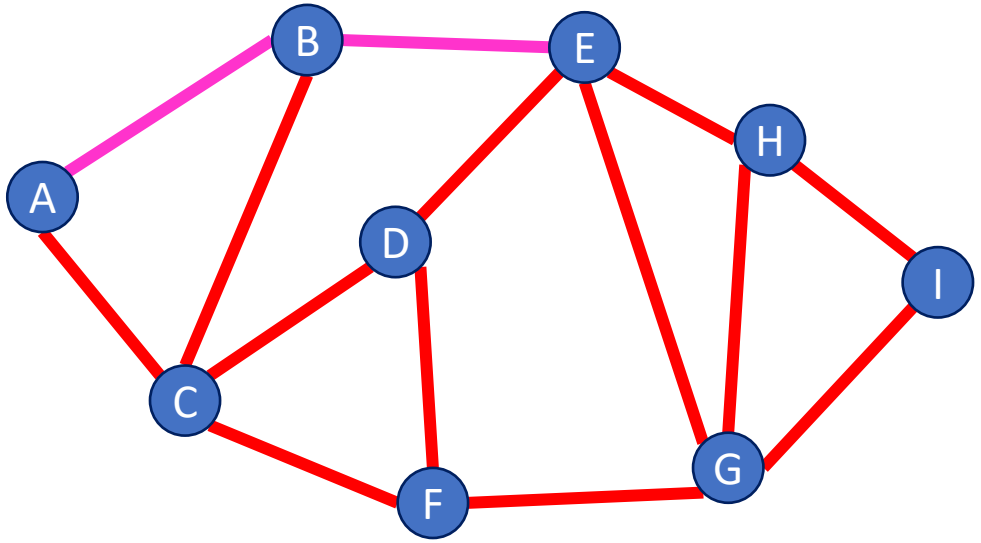
Given an unweighted graph, start node s and an end node t, how long is *shortest path* from s to t?



ShortestPath-Search(G, **A**, **E**) = 2

Shortest Path

ShortestPath(G, s, t, k):
1 iff there is a path from s to t in G with $\leq k$ steps



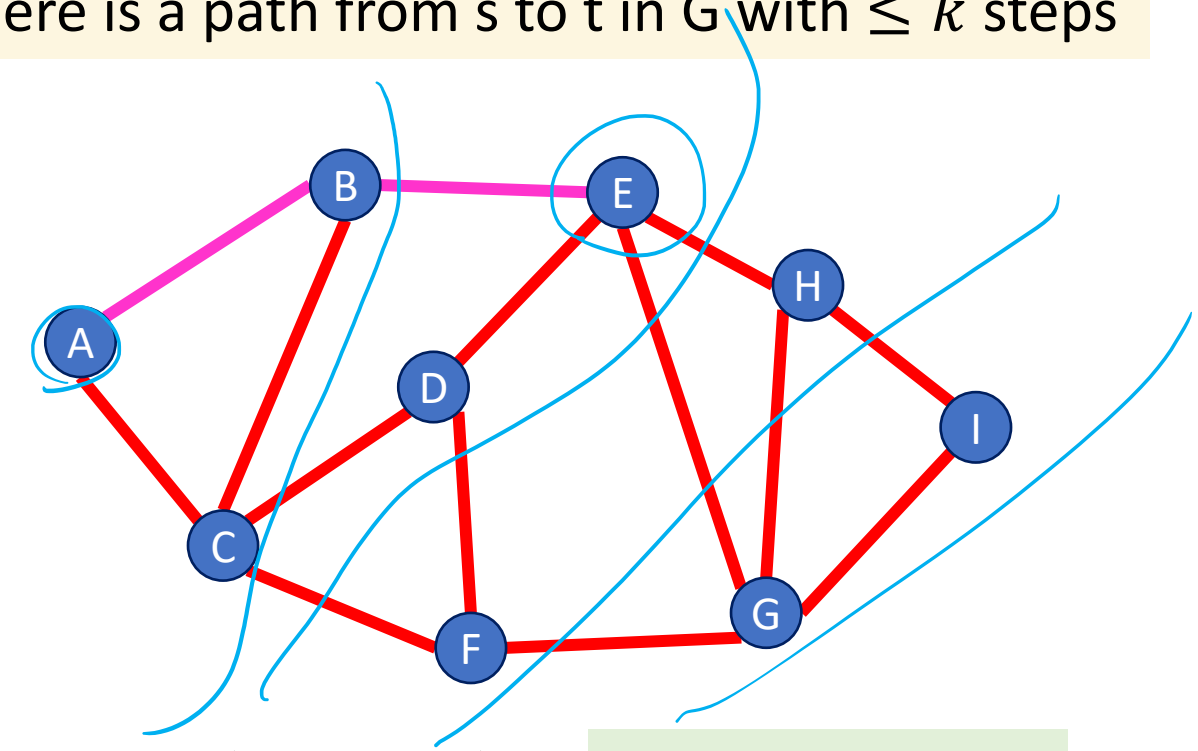
ShortestPath-Search(G, **A**, **E**) = 2

YES

Is *ShortestPath* in **P**?

Breadth First Search

ShortestPath(G, s, t, k):
1 iff there is a path from s to t in G with $\leq k$ steps



Running time: $O(|V| + |E|)$

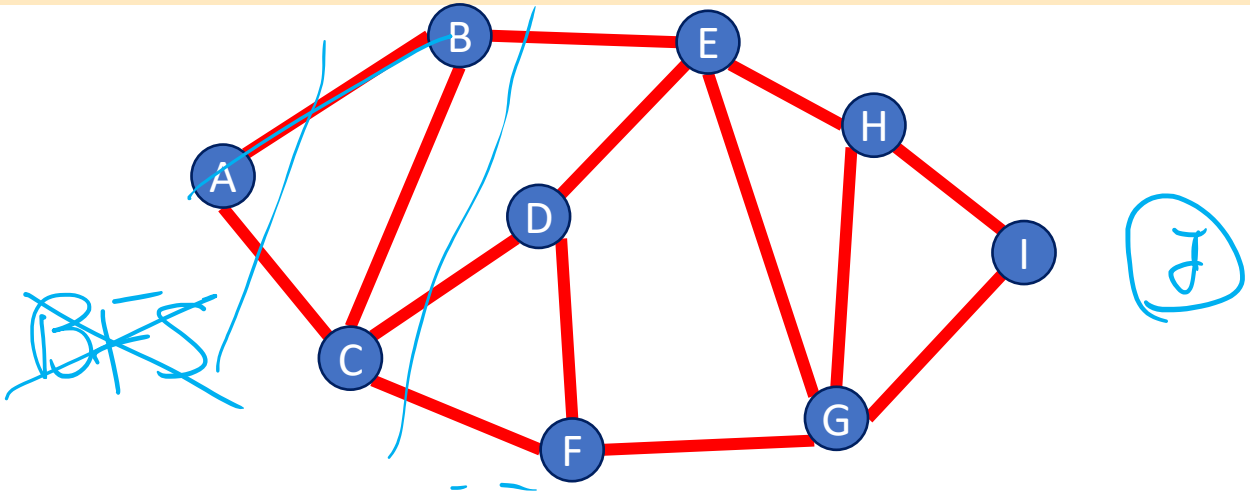
ShortestPath $\in$ **P**

Definition: a ***path*** from v_1 to v_k in a graph $G = (V, E)$ is a sequence of nodes $(v_1, v_2, \dots, v_k)$ such that $\forall\ 1 \le i \le k-1$, $(v_i, v_{i+1}) \in E$. A ***simple path*** is a path with no repeated nodes.

Longest Path

Given a start node s and an end node t, how long is longest *simple* path from s to t?

LongestPath-Search(G, s, t)
Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$
Output: The length of the longest simple path from s to t in G.



Longest Path Algorithm

?

Longest Path Algorithm

maxlength = 0

For all sequences p of distinct vertices in V:

If p is a path from s to v:

if len(p) > maxlength: maxlength = len(p)

return maxlength

∞

$m! \approx 2^{m \log m}$

$|V|!$ $\gg$ n^c all c

What's the running time?

Functions in P?

LongestPath

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.
Output: If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

Definition: a *simple path* in a graph $G = (V, E)$ from $s, t \in V$ is a path from s to t where no node is repeated.

Functions in P?

LongestPath

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.
Output: If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

LongestPath is **not known** to be in **P**: it might be, it might not be. (We'll see a more about this in future classes…)

Definition: a *simple path* in a graph $G = (V, E)$ from $s, t \in V$ is a path from s to t where no node is repeated.

Class EXP:
Exponential Time

"Exponential Time": EXP

P = $\bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$

EXP = $\bigcup_{c \in \mathbb{N}} TIME_{TM}(2^{n^c})$

$2^n, \; 2^{\sqrt{n}}, \; 2^{n^2}$

P $\subsetneq$ **EXP**

(Is **P** a *strict* subset? See Section 13.5/Theorem 13.9)

$F \notin TIME(t) \quad F \in TIME(t \log t)$

Like **P**, **EXP** is also robust to Turing Machine, RAM, or Python computation models

Is *LongestPath* in EXP?

P = $\bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$

EXP = $\bigcup_{c \in \mathbb{N}} TIME_{TM}(2^{n^c})$

LongestPath

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.

Output: If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

Is *LongestPath* $\in$ EXP?

LongestPath-Search

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$

Output: The length of the longest simple path from s to t in G.

Definition 13.2 (P and EXP)

Let $F : \{0,1\}^* \to \{0,1\}$. We say that $F \in \mathbf{P}$ if there is a polynomial $p : \mathbb{N} \to \mathbb{R}$ and a Turing machine M such that for every $x \in \{0,1\}^*$, when given input x, the Turing machine halts within at most $p(|x|)$ steps and outputs $F(x)$.

We say that $F \in \mathbf{EXP}$ if there is a polynomial $p : \mathbb{N} \to \mathbb{R}$ and a Turing machine M such that for every $x \in \{0,1\}^*$, when given input x, M halts within at most $2^{p(|x|)}$ steps and outputs $F(x)$.

How can we turn *LongestPath* into a *decision* problem?

LongestPath-Search

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$

Output: The length of the longest simple path from s to t in G.

$$F: \{0,1\}^* \to \{0,1\}$$

How can we turn *LongestPath* into a *decision* problem?

LongestPath-Search

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$

Output: The length of the longest simple path from s to t in G.

$$F: \{0,1\}^* \to \{0,1\}$$

LongestPath-Decision(G, s, t, ℓ)

Input:

A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.

Output:

If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

Is *LongestPath-Decision* easier than *LongestPath-Search*?

LongestPath-Decision(G, s, t, ℓ):

if *LongestPath-Search* $(G, s, t) \geq \ell$:

return 1

return 0

LongestPath-Decision is easier or equal to *LongestPath-Search*

Can we use *LongestPath-Decision* to solve *LongestPath-Search*?

LongestPath-Search (G, s, t)

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$

Output: The length of the longest simple path from s to t in G.

LongestPath-Decision (G, s, t, ℓ)

Input: A finite graph $G = (V, E)$, two vertices, $s, t \in V$, and a path length, $\ell \in \mathbb{N}$.

Output: If there is a simple path from s to t in G of length at least ℓ, 1. Otherwise, 0.

Search(G, s, t)

for $(\ell = \cancel{0, \ldots} |V| \ldots 0)$

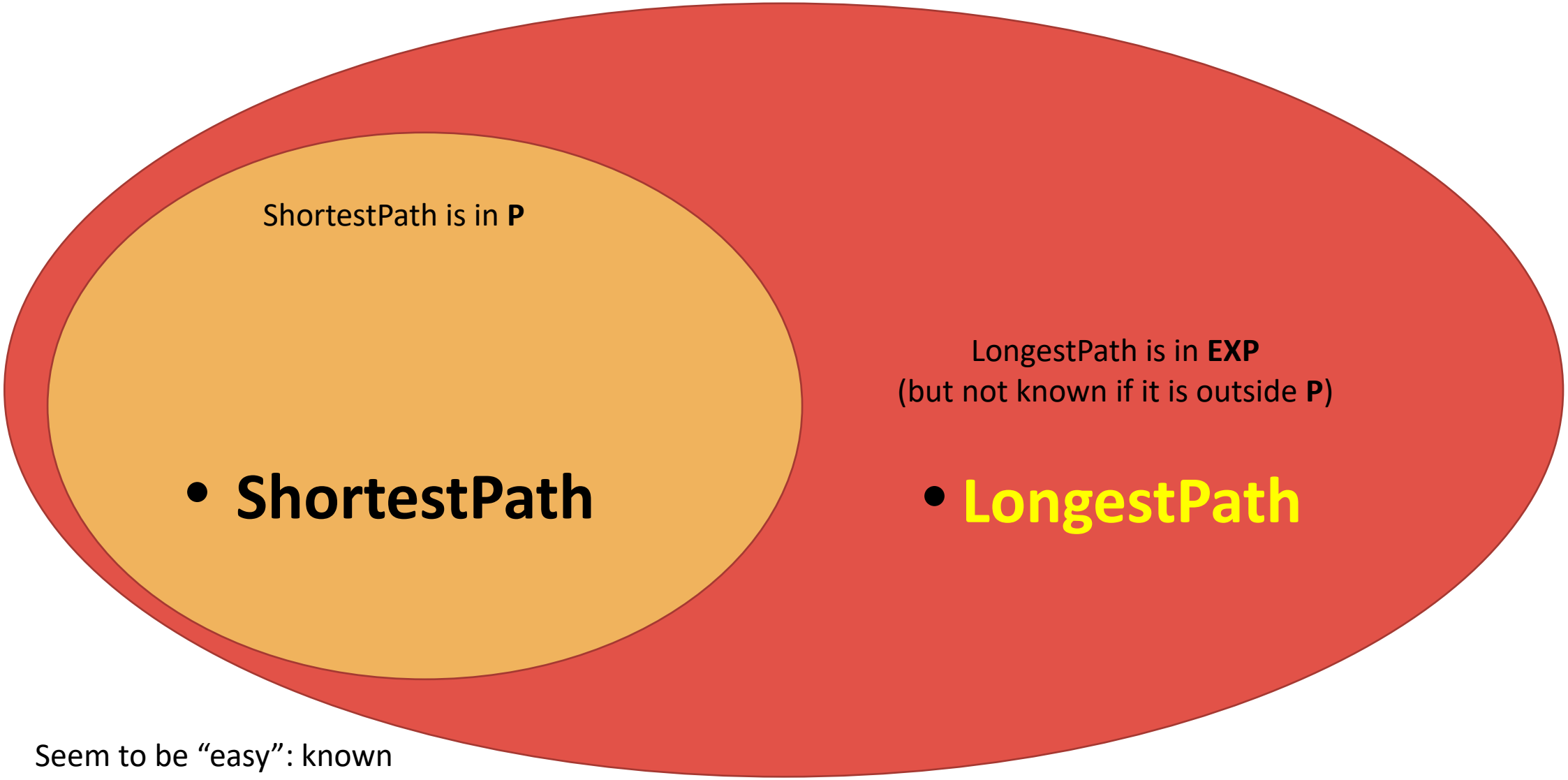
if Decision$(G, s, t, \ell) = 1$

output ℓ.

Search and *Decisional LongestPath* are "equivalent in poly-time"

LongestPath-Search ∈ **P** iff *LongestPath-Decision* ∈ **P**

LongestPath-Search ∈ **EXP** and *LongestPath-Decision* ∈ **EXP**



Seem to be "easy": known polynomial-time algorithms

Class **P** = $\bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(n^c)$

Class **EXP** = $\bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(2^{n^c})$

Logistics

- Mead Coffee/Food
  Today (Apr 9) 2:30pm, Einstein Bros. Bagels

HW 8 coming soon.

Quiz 11 due next Monday.
Midterm 3: next Tue, Apr 14, 9:30am.

Plan

Polynomial-time Reductions

Textbook [TCS] Section 14

https://introtcs.org/public/lec_12_NP.html
Polynomial-time reductions